

AGILE: Elastic Distributed Resource Scaling For Infrastructure-as-a-Service

Anirudh Kaluri, Sagar Dama

CSC 724: Advanced Distributed System
Dr. Helen Gu
April 23, 2024

INTRODUCTION

AGILE

1) Host:

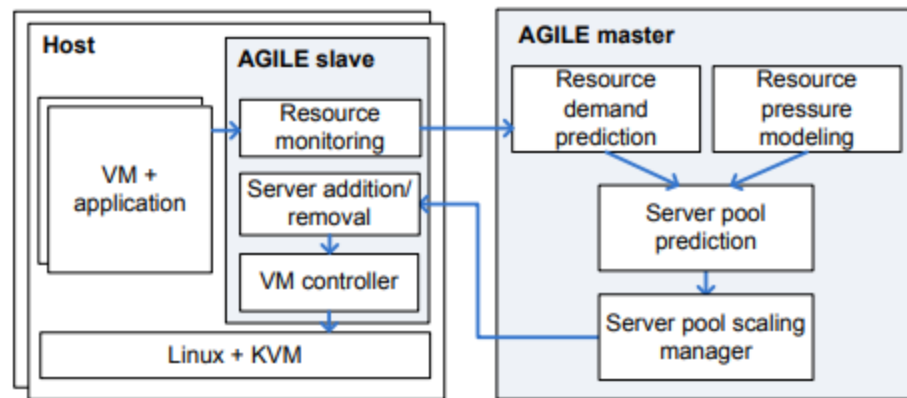
Runs Virtual machines (VMs) with applications.
Managed by AGILE slave and Linux + KVM.

1) AGILE Slave:

Resource Monitoring: Continuously monitors VM resource usage (CPU, memory, etc.).
VM Controller: Handles VM addition/removal and cloning.

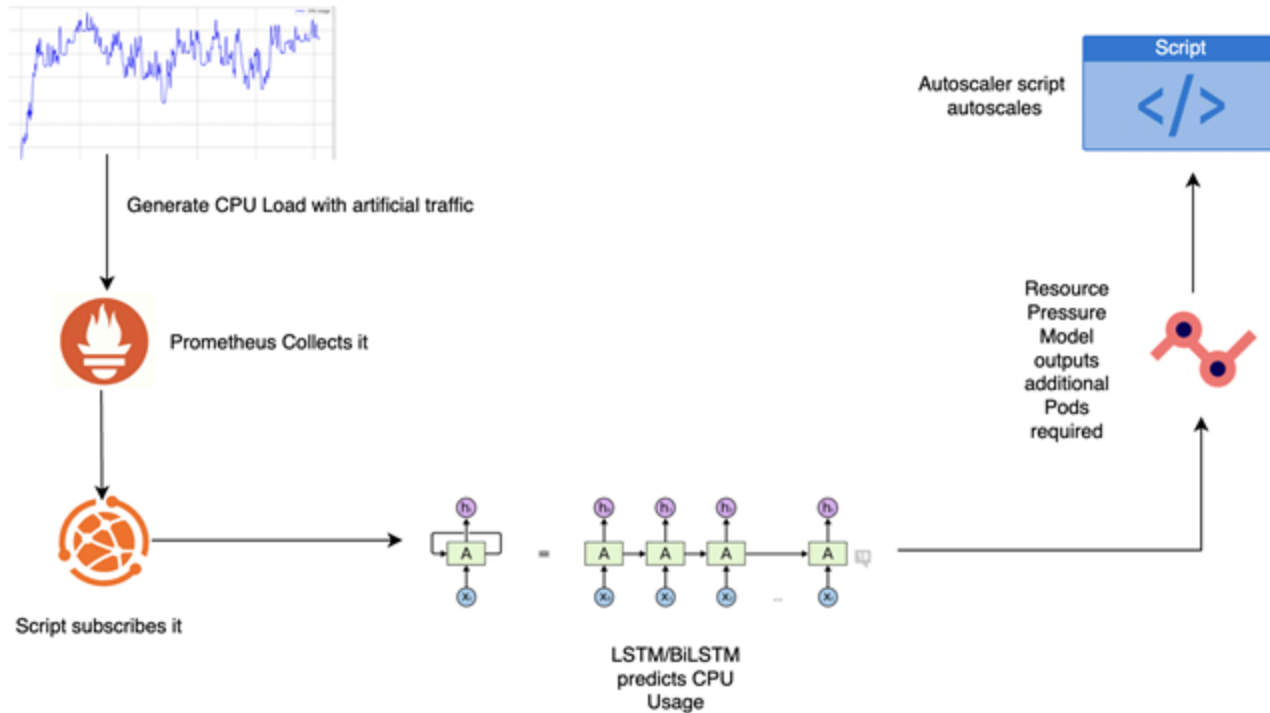
1) AGILE Master:

Resource Demand Prediction: Uses wavelet transforms for medium-term predictions.
Resource Pressure Modeling: Maps resource pressure to SLO violation rates.
Server Pool Scaling Manager: Manages server pool scaling based on predictions.

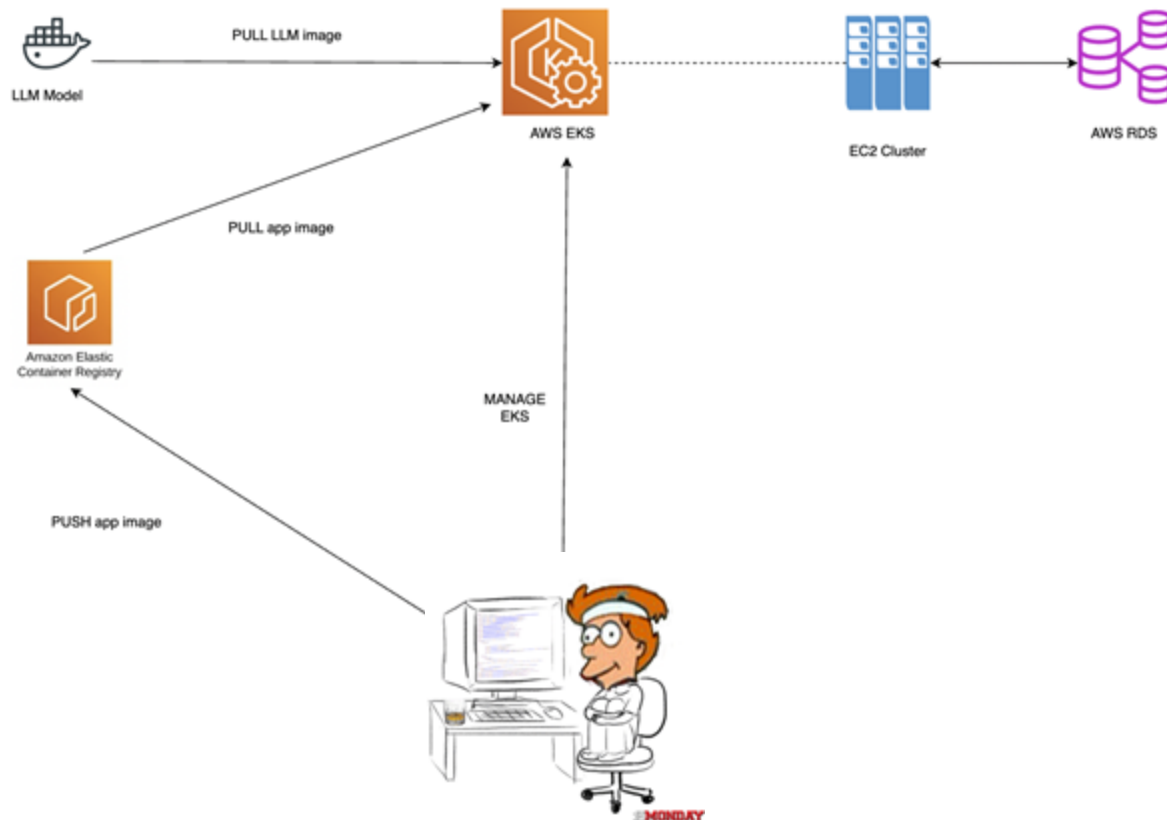


OUR WORKFLOW, DESIGN AND DEPLOYMENT

Workflow



Our System Design

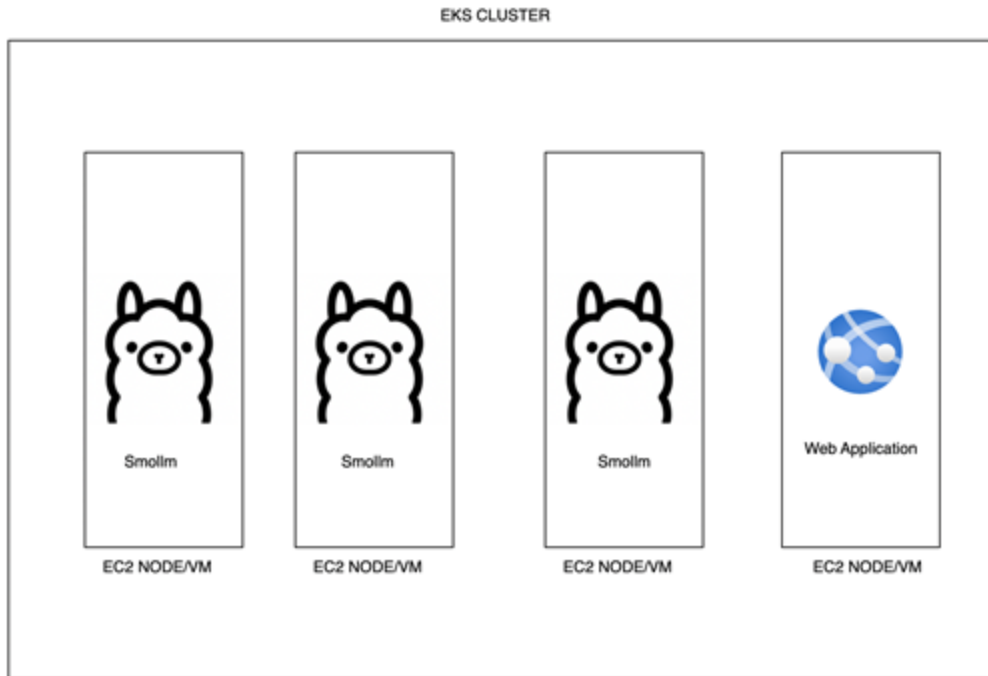


A closer look into our EC2

Type: M6l.xLarge, 4vCPUs, 16GB Memory

LLM: Smollm-135m

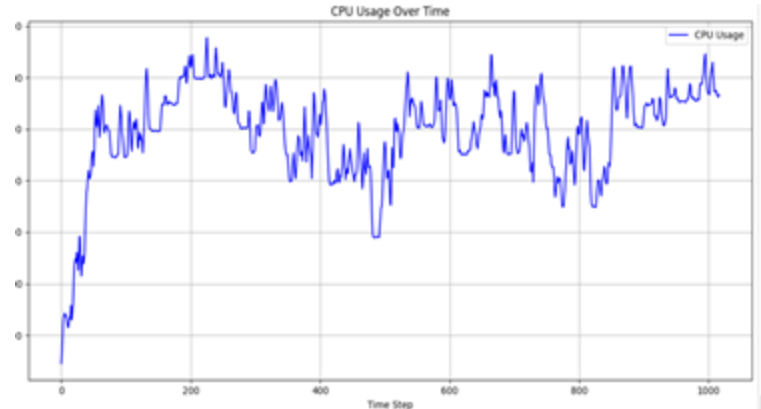
Web Application: Django Service, Gunicorn workers



EXPERIMENT

Artificial Traffic Generation

1. Locust to generate the Traffic
2. Custom Phase wise Rules
3. System Meta-Data
4. 7 LLM pods
5. 1 Web Server
6. Simulation period of 3 hrs



```
stages = [
```

```
{ "duration": 180 * 1, "users": 2, "spawn_rate": 1 }, # ●
{ "duration": 180 * 2, "users": 4, "spawn_rate": 1 }, # ▲
{ "duration": 180 * 3, "users": 8, "spawn_rate": 2 }, # ▲
{ "duration": 180 * 4, "users": 12, "spawn_rate": 2 }, # ●
{ "duration": 180 * 5, "users": 0, "spawn_rate": 1 }, # ▼
{ "duration": 180 * 6, "users": 3, "spawn_rate": 1 }, # ●
```

- 1) Multiple phases spread across 3 hrs with each phase lasting over 3 mins
- 2) Each Phase has 3 parameters.
 - a) duration - 3 mins
 - b) users: no of users which will be accessing the resource
 - c) spawn rate: rate at which the users are spawned until they reach to the desired users count

Future CPU Prediction

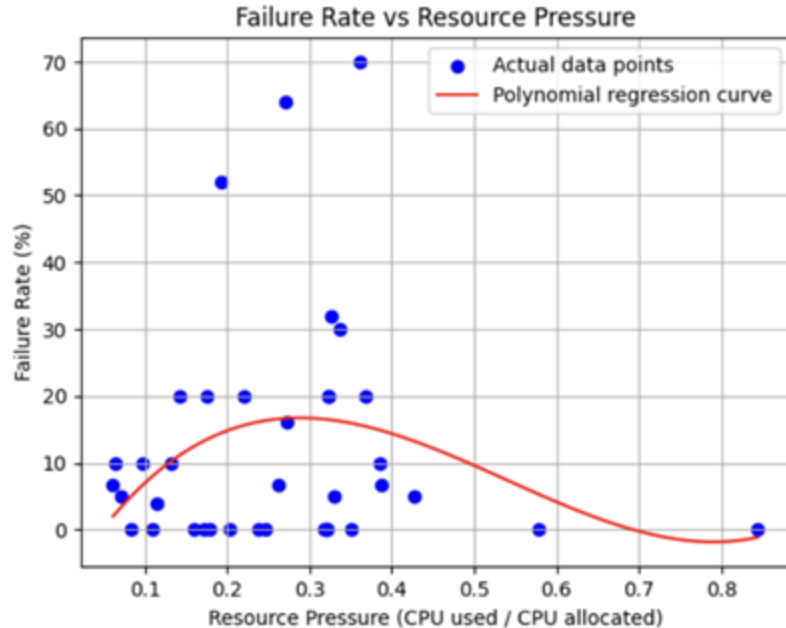
- 1) Models Chosen: LSTM, BiLSTM
- 2) Metrics used to test and validate: RMSE
- 3) Hyperparameters
 - a) Optimizer: Adam
 - b) Learning Rate: 0.001
 - c) Loss: MSE(Mean Squared Error)
 - d) Epochs: 100
 - e) Batch Size: 32

FIGURE 1: DEQUENT LAYER

Layer (type)	Output Shape	Param #
lstm_40 (LSTM)	(None, 20, 256)	264,192
dropout_50 (Dropout)	(None, 20, 256)	0
lstm_41 (LSTM)	(None, 20, 256)	525,312
dropout_51 (Dropout)	(None, 20, 256)	0
lstm_42 (LSTM)	(None, 20, 128)	197,120
dropout_52 (Dropout)	(None, 20, 128)	0
lstm_43 (LSTM)	(None, 64)	49,408
dropout_53 (Dropout)	(None, 64)	0
dense_30 (Dense)	(None, 256)	16,640
dropout_54 (Dropout)	(None, 256)	0
dense_31 (Dense)	(None, 128)	32,896
dense_32 (Dense)	(None, 10)	1,290

Total number of parameters: 1,000,000

Resource Pressure Model



Predicted Resource Pressure= CPU Predicted / CPU Currently Allocated

Predicted Failure Rate = $f(\text{Predicted Resource Pressure})$

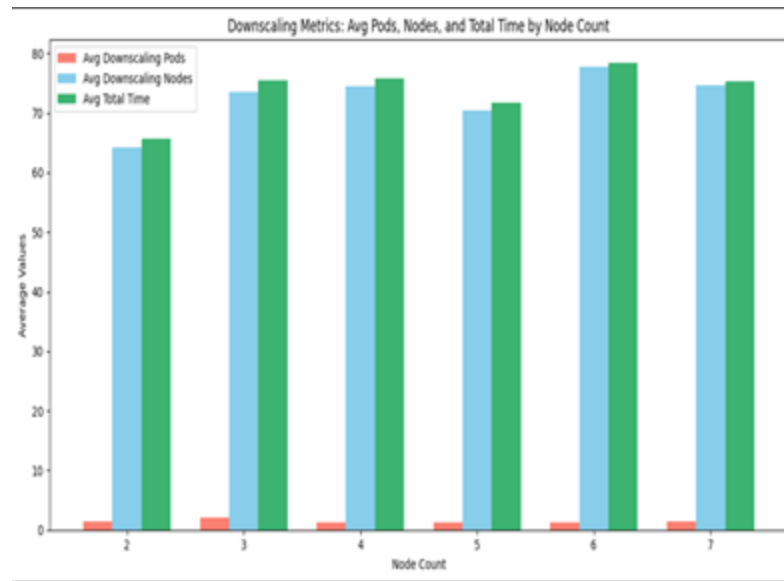
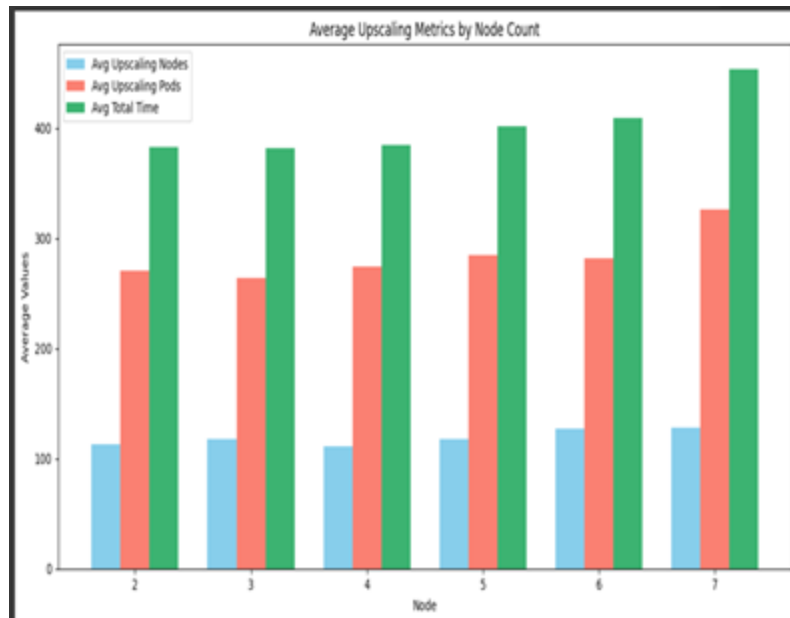
If High_Resource_Pressure:

$0.206 = \text{CPU Predicted} / \text{Required CPU}$

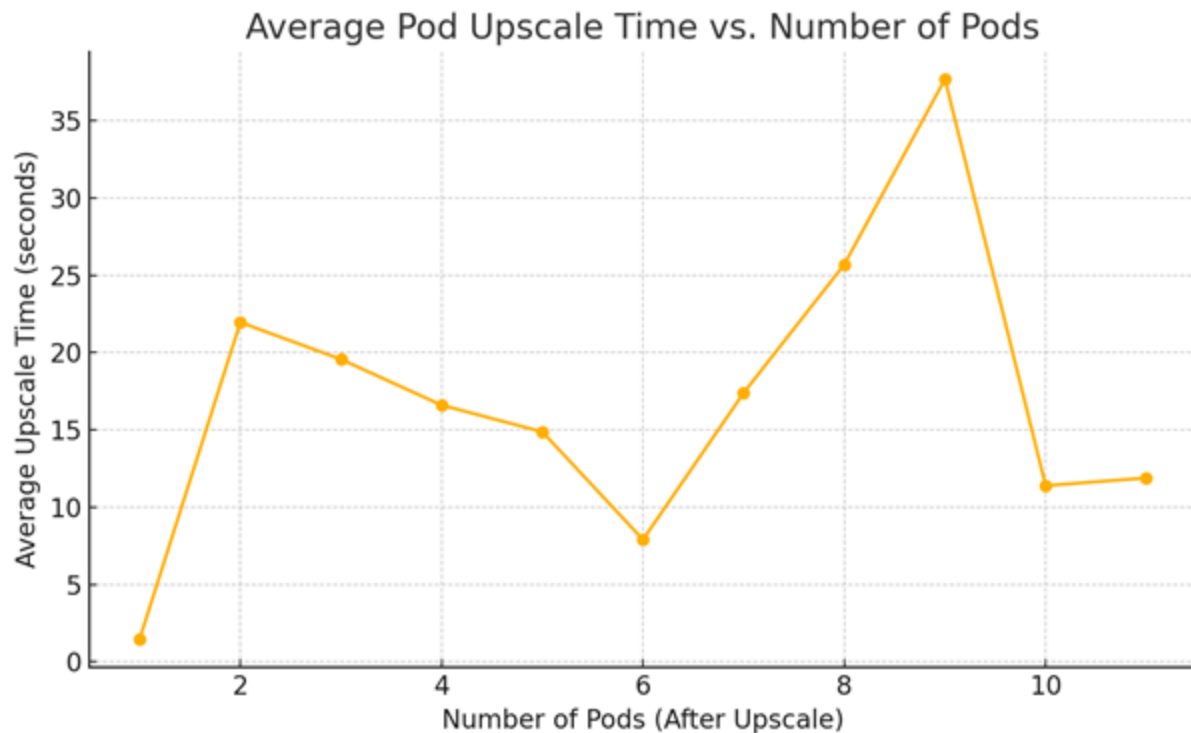
$\text{Required CPU} = \text{CPU Predicted} / 0.206$

Scale

Lead Time Prediction including Nodes

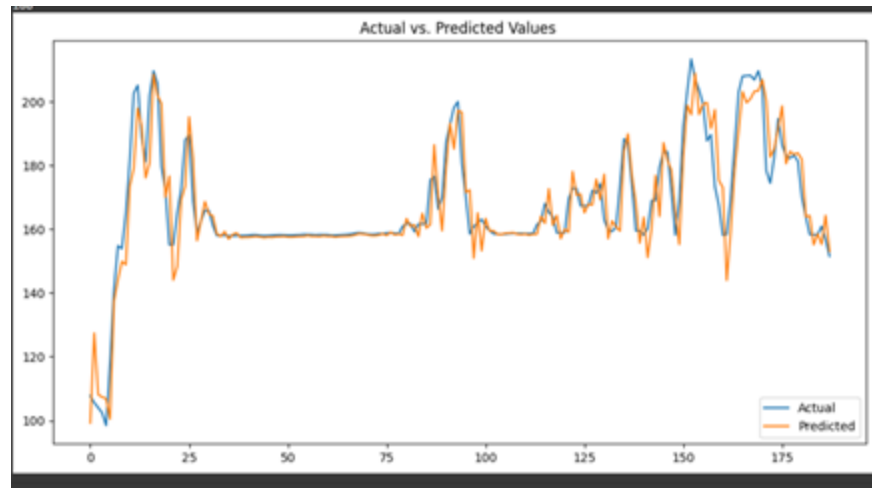
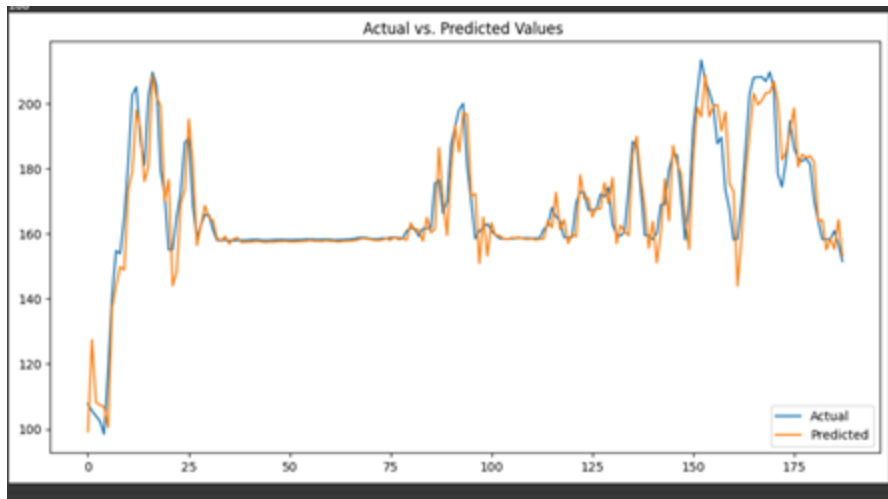


Lead Time Prediction using just Pods (Constant 8 Nodes)



RESULTS

Prediction Performance on Training Data



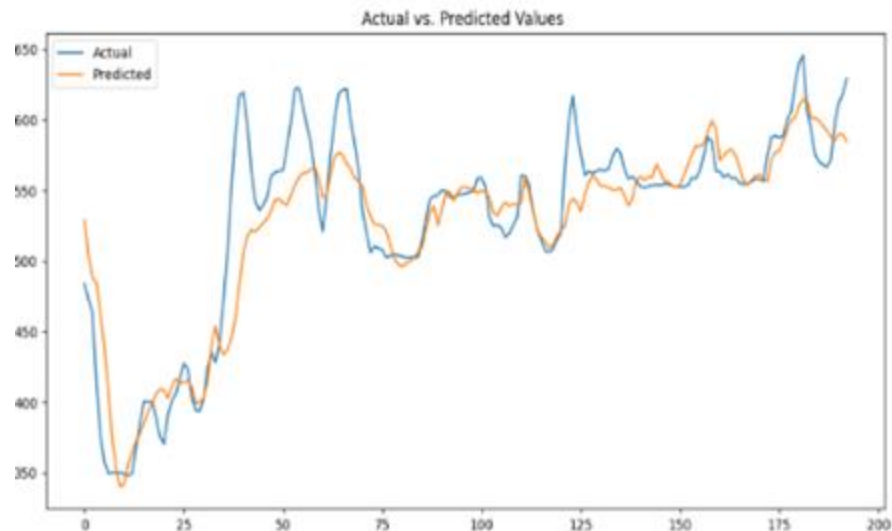
Prediction Performance on Training Data



Prediction Performance with LSTM on Testing Data

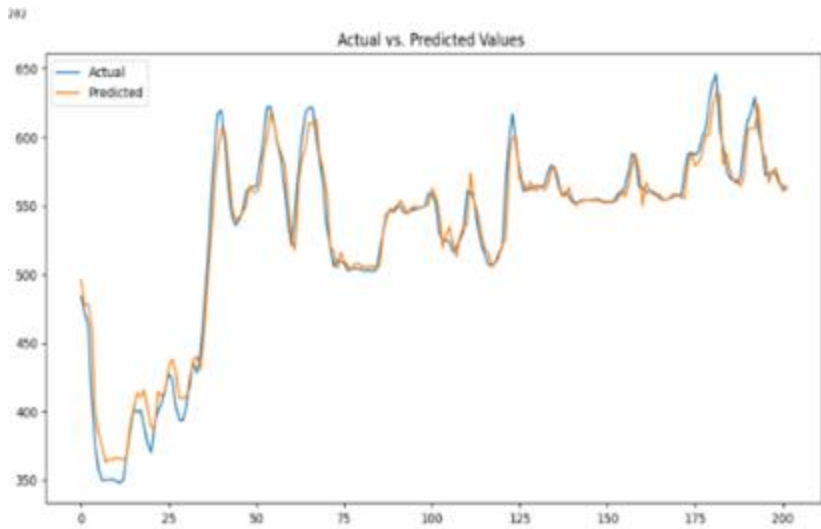


6-Input Window 3 Future steps

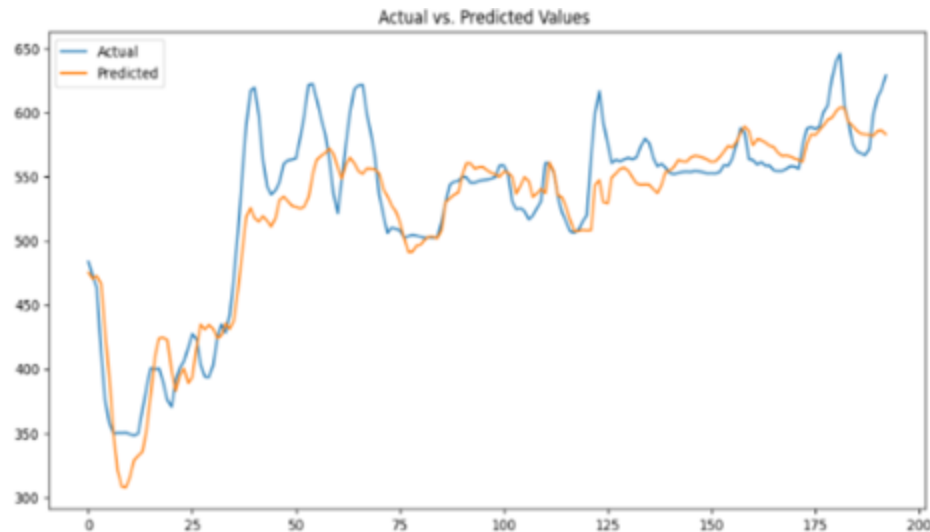


20-Input Window 10 Future steps

Prediction Performance with Bi LSTM on Testing Data



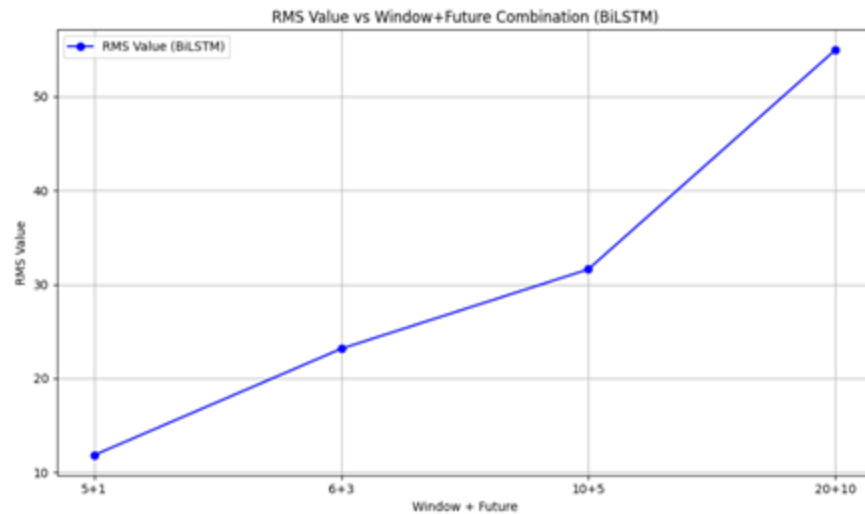
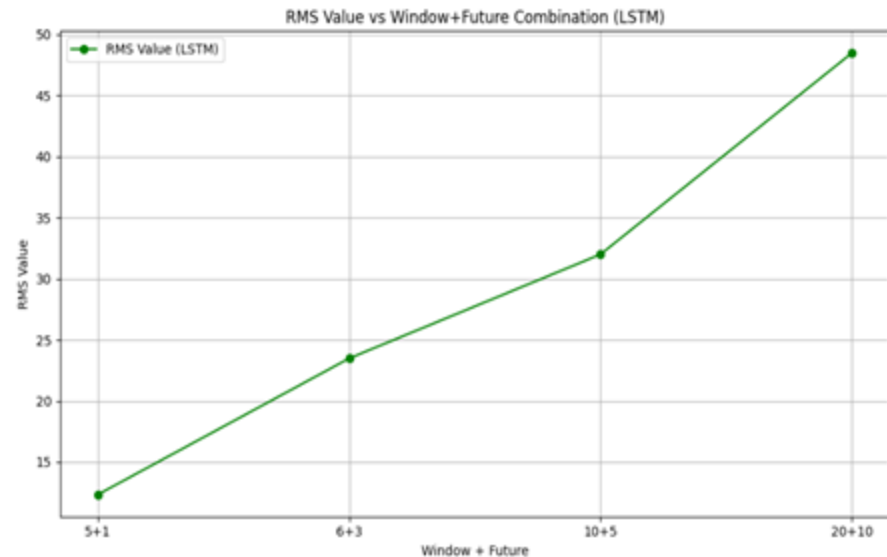
6-Input Window 3 Future Steps



20-Input Window 10 Future Steps



RMSE For Different Windows



CHALLENGES

Prometheus going down when downscaling

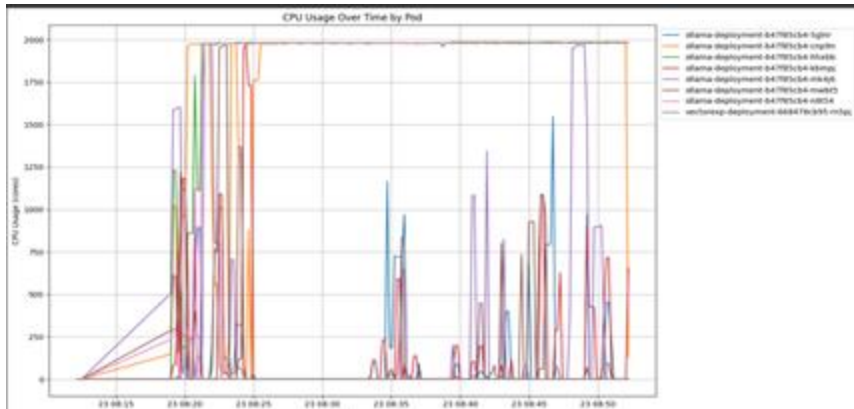
Request Time Out Error

Lead Time Calculation

Load-Balancing amongst LLM Pods

Recreating traffic trends

Autoscaling conflict



Takeaways

Tackling real-world problems by planning

Building research experiment

Lot of learning: Kubernetes, AWS, Load Balancing, Autoscaling

Thank you

Q/A